

VentaGo 현황 진단서

Phase 64 이후 재평가

무엇이 해결됐고, 무엇이 아직 남았는가 — 코드 실측 기반

문서 성격	2026-07-27 비교분석 보고서의 후속 재평가 . 그 사이 수행된 보완 작업(Phase 64 W1~W9, 권한 가드 전개, 스키마 제약 정합화)을 반영해 잔여 과제를 재산정한다.
평가 기준일	2026-07-29 (직전 커밋 <code>b02ac0d</code> , 2026-07-28 14:43 -03:00)
평가 대상	<code>ACE_online_1.0</code> 모노레포 — <code>api-ventago</code> (NestJS) · <code>ventago-app</code> (Next.js) · 모바일/에이전트 앱 · PostgreSQL 18
근거	소스 코드 직접 실측(파일·라인 확인), 마이그레이션 SQL 218개, Phase 63/64 계획·검증·부하테스트 문서, 빌드·정적분석 재실행 결과
평가 범위	재고 정합성 · 멀티테넌트 격리 · 보안 통제 · 가용성/DR/관측 · 계획·분석 기능
용도	내부 의사결정용 — 다음 Phase 범위 확정

읽는 법. 이 문서는 칭찬이 아니라 잔여 부채 목록이다. 지난 이들의 작업은 실제로 큰 성과를 냈고 그 부분은 3장에 근거와 함께 명시했다. 그러나 4장 이후는 **아직 고쳐지지 않은 것만** 다룬다. 모든 주장에는 파일·라인 근거를 붙였고, 코드로 확인하지 못한 것은 "추정"으로 표시했다.

01 경영진 요약

결론. 지난 이틀간의 보완으로 "판매 한 건이 두 건이 되는" 부류의 치명 결함은 실제로 닫혔다. 스테이징 부하시험에서 40매장 6,435건 판매에 저장 실패·중복 채번·부분 저장이 전부 0으로 실측됐다. 그러나 **재고 정확성의 근본 문제는 닫히지 않았다** — 원장 불변 규칙이 파일 한 개에만 적용됐고, 캐시와 원장을 대조·보정하는 장치는 여전히 코드에 존재하지 않는다. 여기에 이번 실측에서 **이전 보고서에 없던 위험 3건**(감사로그 크로스테넌트 조회, 사용자 관리 경계 미검증, 커밋된 자격증명)이 새로 확인됐다.

닫힌 치명 결함

4 / 4

판매 먹통·취소 원자화·생산 완료·outbox lease — 코드·유닛·동시성·부하까지 검증

남은 정확성 부채

8건

재고 원장·캐시 이중진실 계열. 자동 대조 장치 0건

신규 발견 위험

3건

감사로그 전매장 조회·사용자 경계 미검증·커밋된 DB 비밀번호

부하 여유

6배

요구 5건/s 대비 29건/s 실측, P95 79ms (목표 300ms)

역량 성숙도 재평가 (5점 만점)

회색 = 2026-07-27 평가, 청록 = 2026-07-29 재평가.

기능 범위		4.6 → 4.6
현장 적합성		4.7 → 4.7
확장·통합성		4.2 ↑ 4.0
거래 안전성 신설		4.2 신규
데이터 정확성		3.1 ↑ 2.4
보안·감사		2.9 ↓ 3.2
분석·예측		3.3 ↓ 3.4
가용성·복구		2.7 → 2.7

점수가 내려간 두 항목의 이유. 보안·감사(3.2→2.9) — 이전 평가는 "응용 보안 강화"에 근거했으나, 이번 실측에서 감사로그가 매장 경계 없이 조회되고 사용자 수정 경로에 테넌트 검증이 없는 것이 확인됐다. 새 결함이 생긴 것이 아니라 **기존 결함이 이번에 발견된 것이다**. 분석·예측(3.4→3.2) — 재고 부족 알림이 상품 무관 고정 임계값 5로 하드코딩돼 있어, 계획 기능의 실질 수준이 이전 평가보다 낮다.

향후 12개월 전략 — 이전 권고 대비 변경점

기간	2026-07-27 권고	2026-07-29 수정 권고
0~3개월	판매 먹통성, 취소·생산·보류 원자성, 재고 동시성	대부분 완료 → 재고 원장 단일화와 테넌트·감사 경계로 교체. 남은 것은 브라우저 UAT와 Phase 64 W10.
3~6개월	outbox lease, DR 훈련, 관측·알람, 테넌트 DB 제약	outbox lease는 완료. DR·관측·알람은 그대로 유효하며 우선순위가 올라갔다 — 장애를 감지할 수단이 현재 없다. 테넌트 DB 제약(RLS)은 도입 금지로 이미 결정됐으므로 애플리케이션 계층 강제로 대체한다.
6~12개월	수요예측, 자동발주, 고객 세그먼트, 생산계획	변경 없음. 단 재고 캐시가 정확해지기 전에는 자동발주를 켜면 안 된다 — 잘못된 잔액으로 발주를 내게 된다.

02 지난 이틀의 보완 작업 — 실제로 무엇이 달했나

이전 보고서가 지목한 PO 4건이 모두 코드로 구현되고 운영 DB까지 적용됐다. 근거는 문서가 아니라 실행 결과다.

이전 지적 (PO)	상태	확인된 증거
판매 먹등·거래 영수증 키 중복 판매/이중 차감	해결	<code>sale_idempotency_keys</code> 테이블 + <code>uq_sale_idem_store_key</code> UNIQUE, 로컬·운영 양쪽 적용 완료. 스테이징 실증: 동일 <code>Idempotency-Key</code> 2회 요청 → 같은 <code>saleId=181974</code> 반환, 판매 행 1건. <code>claim INSERT</code> 가 판매와 같은 트랜잭션 안에 있어 "판매는 있는데 키가 없다"가 구조적으로 불가능.
원자적 취소·생산·보류판매 부분 저장·이중 처리	해결	취소·보류 <code>create/update/remove</code> ·생산 완료를 각각 단일 트랜잭션 + <code>FOR UPDATE</code> 재검사로 재작성. 동시성 스위트에서 동일 문서 병렬 취소 5요청 → 성공 1건·역분개 1건(20회 반복) 통과. 생산 완료는 존재하지 않는 <code>stocks.productId</code> 참조를 <code>productBranchId</code> 원장 패턴으로 교체.
재고 조건부 차감 동시 판매 초과 판매	해결	검사와 차감을 한 문장으로 통합 — <code>UPDATE products SET stock=stock-\$1 WHERE id=\$2 AND stock>=\$1 RETURNING id</code> . 0행이면 400. 매장 설정 <code>allowSaleWithoutStock</code> 을 존중해 허용 매장은 회귀 없음(양쪽 갈래 모두 테스트).
outbox lease·자동복구 중복 전송·영구 고착	해결	<code>FOR UPDATE SKIP LOCKED</code> 단일 claim문 + <code>locked_by / lease_expires_at</code> 컬럼 + 만료 회수. 워커 4개 동시 claim → 중복 집행 0, lease 만료분 회수 후 재claim 성공 실측. <code>enqueue</code> 를 판매 트랜잭션 내부로 이동해 "판매 커밋 = outbox 행 존재"가 동치가 됨.

이전 보고서가 요구하지 않았지만 함께 처리된 것

항목	내용과 의미
매장 경계 검증 (R8)	판매 생성의 상품·판매원·지점을 전부 <code>findOne({id, storeId})</code> 로 교체하고 <code>storeId</code> 폴백을 제거(없으면 400). 식당 모드 주문 경로에는 검증이 아예 없던 것을 새로 발견해 추가했다 — 소매 경로만 고쳤다면 경계가 그대로 뚫려 있었을 것이다.
스키마 제약 정합화	로컬·운영 제약 집합을 전수 대조해 125건의 결손 UNIQUE/CHECK/FK를 먹등 마이그레이션으로 보강 (2026-07-28-phase64-missing-constraints.sql, 644줄). 적용 전 위반 검사에서 운영 데이터 위반 0건 확인.
권한 가드 전개	2026-07-28 하루에만 <code>stocks·usuarios·gastos·productos·cuentas·talleres·materia-prima</code> ·참조데이터 등에 <code>@FunctionGuard</code> 를 일괄 적용하는 커밋 8건. 권한 재시드를 id가 아닌 slug 기반으로 전환해 드리프트에 대응.
쓰기 경로 규약 문서화	<code>CLAUDE.md</code> 에 "단일 트랜잭션 원칙 / <code>stocks</code> 는 append-only / 트랜잭션 안 외부 I/O 금지 / 커밋 후 = 성공 / 락 순서 고정" 6개 규약을 명문화. 재발 방지 관점에서 이번 작업의 가장 오래 남을 산출물이다.

부하 실측 결과 (스테이징, 운영 백업 복원본). 40매장 · 25건/s · 2분 → checks 100%(4,694/4,694), HTTP 실패 0%, 저장 6,435건. `daily_number` 중복 0 · 부분 저장 0 · 음수재고(비허용 매장) 0 · outbox 정체 0. P95 79.2ms로 목표 300ms 대비 3.8배 여유. 관측된 최대 초당 판매 29건.

다만 이 성과에는 조건이 붙는다. 위 검증은 전부 SQL 수준 동시성 구성물과 API 계층에 대한 것이다. 브라우저 UAT는 아직 한 번도 실행되지 않았다(Phase 64 요구사항 12건 전부 UAT 칸 공란). 또한 Phase 64의 마지막 Wave(W10 — 스키마 대조:intel 재생성·UAT)가 미완이고, `.planning/ROADMAP.md` 는 여전히 "Plans: 0/10 executed"로 표기돼 실제 상태와 어긋나 있다.

03 최대 잔여 부채 — 재고 원장·캐시 이중 진실

핵심. Phase 64는 재고 원장을 **append-only**로 고정하겠다고 선언했고, 그 검증 게이트(`.destroy()` → 0건)를 통과했다. 그러나 그 게이트는 `stocks.service.ts` 한 파일만 검사했다. 같은 `stocks` 테이블을 UPDATE·DELETE 하는 코드가 다른 파일에 9곳 그대로 남아 있다.

3-1. 원장을 여전히 변경·삭제하는 코드

파일 : 라인	동작	맥락
<code>products/productStock.service.ts:622</code>	destroy	당일 입고 0 처리 (<code>correctTodayStocks</code>)
<code>products/productStock.service.ts:629</code>	destroy	잔여 행 정리
<code>products/productStock.service.ts:914</code>	destroy	<code>deleteStockTodayByParent</code> — 활성 라우트 <code>products.controller.ts:414</code>
<code>products/productStock.service.ts:1493</code>	destroy	<code>editMadreVariants</code> 색상 삭제
<code>products/productStock.service.ts:401</code> / 483 / 727	update	원장 행의 수량을 절대값으로 덮어씀
<code>products/products.service.ts:431</code>	update	<code>stockRecord.update({stock})</code>
<code>subcon/subcon-material-issues/subcon-material-issue.service.ts:59</code>	update	외주 자재 출고

DB 계층 강제(트리거·권한 분리)도 없어, 규약은 코드 리뷰로만 지켜진다.

3-2. 원장만 쓰고 잔액 캐시를 갱신하지 않는 경로 — 드리프트 발생원

경로	결과
이동(movido) / 폐기(fallado) <code>stocks/stocks.service.ts:368, 402</code>	<code>Sale + SaleItem + Stocks</code> 만 INSERT하고 <code>products.stock</code> 은 건드리지 않는다. 지점 간 이동·폐기 1건마다 캐시와 원장이 영구히 벌어진다. 재고 조정 화면에서 가장 자주 쓰이는 경로다.
외주 로트 입고 <code>products/productStock.service.ts:89</code> (<code>ingresarStockPorMatrix</code>)	원장에만 기록. 생산 완제품 입고가 POS 판매 검증(<code>product.stock</code> 기준)에 반영되지 않는다.
변형상품 일괄 생성 <code>productStock.service.ts:265-333</code>	자식 상품 <code>create</code> 에 <code>stock</code> 필드가 없어 자식 캐시가 0에서 출발. 게다가 부모 집계(:358-372)가 원장 합이 아니라 자식 캐시 합 이라 오차가 부모로 전파된다.

3-3. 대조·보정 장치가 없다

재고 관련 크론·재계산 잡·대조 엔드포인트가 코드베이스에 0건이다. 유일한 대조 수단은 docs/db-risk-analysis-20260727.md 에 적힌 수동 SQL 한 개다. 같은 문서가 "products.stock 드리프트 실재"를 운영 실측으로 기록하고 "야간 보정 잡 + 1회 백필"을 계획했으나 구현되지 않았다.

이식 비용은 낮다. 동일한 형태의 장치가 이미 mercadopago/cron/mp-wallet-reconcile.cron.ts:27-65 에 존재한다 — 지갑 잔액 캐시와 계산값의 drift를 탐지해 자동 보정한다. 재고에 같은 패턴을 복제하면 된다.

3-4. 이번 실측에서 새로 확인된 두 가지

항목	내용
이동 유형 값이 모델 정의 밖	모델 union은 'sale' 'adjust' 'suspend' (stocks.model.ts:15)인데, 실제로는 'ajuste' (stocks.service.ts:151 — Phase 64가 새로 도입한 보정 행)와 'produccion' (work-order.service.ts:209, 232)이 기록된다. 재고 코크핏 리포트는 type NOT IN ('adjust', 'suspend') 로만 필터하므로 이 두 유형이 정상 입고·판매로 집계된다. 즉 Phase 64가 도입한 보정 행 자체가 리포트를 오염시킨다. 직접 확인: reportsStocksCockpit.service.ts:476, 479, 723, 726
가용재고(ATP) 개념 부재	예약은 원장의 type='suspend' 음수 행으로만 표현되고 products.stock 에는 반영되지 않는다. 결과적으로 두 저장소의 의미가 다르다 — 캐시는 물리 현재고, 원장 합은 현재고-예약. 같은 "재고"를 계산하는 SQL이 최소 4벌이고 is_active · type 필터가 서로 다르다. 결정적으로, 문서에 적힌 그 대조 SQL은 예약분을 걸러내지 않아 예약이 있는 상품을 전부 드리프트로 오탐한다.

3-5. 음수 재고 방어 적용 범위

조건부 차감이 적용된 곳은 POS 판매 경로 하나뿐이다. 이동·폐기·조정·생산 자재 소비·온라인 hold·보류 예약·절대값 정정 경로에는 재고 검사가 없다. (매장 설정상 음수 재고를 허용하는 것이 기본 정책이므로 이것이 전부 결함은 아니지만, 비허용 매장에서조차 판매 외 경로로는 음수가 만들어진다는 점은 정책과 어긋난다.)

근거: 코드 실측 — stocks.model.ts , stocks.service.ts , productStock.service.ts , work-order.service.ts , reportsStocksCockpit.service.ts , docs/db-risk-analysis-20260727.md

04 멀티테넌트 격리와 보안 통제

신규 발견 — 이전 보고서에 없던 항목. Phase 64가 판매 경로의 매장 경계를 막았지만, 감사 로그와 사용자 관리 경로는 여전히 열려 있다. 아래 두 건은 이번 진단에서 직접 코드로 확인했다.

경로	심각도	확인된 내용
<code>GET /audit-log/entity/:entityType/:entityId</code> audit-log.controller.ts:37-42	높음	핸들러가 요청자 정보를 아예 받지 않는다(@GetUser 없음). entityType·entityId만으로 조회되므로 다른 매장의 감사 로그를 그대로 읽을 수 있다.
<code>GET /audit-log/store</code> audit-log.controller.ts:50-53	높음	<code>if (!user.roles roles.length===0 roles[0]==null) return getAllLogs(page, size, {})</code> — 역할 배열이 비면 필터 없이 전 매장 감사로그를 반환한다. 권한 실패가 차단이 아니라 전체 공개로 귀결되는 구조다.
<code>PUT /users/:id</code> users.service.ts:299, 322	높음	<code>findByPk(id)</code> 로 대상을 찾을 뿐 storeId 대조가 없다. 게다가 <code>if (dto.storeId !== undefined) updateData.storeId = dto.storeId</code> — 사용자를 임의의 다른 매장으로 옮길 수 있다. <code>DELETE /users/:id</code> 도 같은 문제(자기 계정 차단만 존재).
커밋된 DB 자격증명	높음	마이그레이션 SQL 10개 파일의 주석에 <code>PGPASSWORD</code> 가 평문으로 들어 있고 전부 git에 추적·커밋된 상태다. (본 문서에는 파일 목록만 남기고 값은 옮기지 않았다.) 해당 계정 비밀번호 회전이 필요하다.

4-1. DB 계층 테넌트 강제 — 없음, 그리고 "도입 금지"로 결정됨

RLS 정책 0건, store_id를 포함한 복합 FK 0건, 그 전제인 `UNIQUE(store_id, id)` 0건이다. 이전 보고서는 "복합 FK/RLS로 이중 방어"를 권고했으나, 그 사이 RLS는 명시적으로 도입 금지가 결정됐다 — 근거는 pgbouncer transaction pooling 환경에서 세션 변수 기반 RLS가 오히려 테넌트 누출 위험을 만들고 superadmin 크로스매장 기능을 파손한다는 것이다. 이 결정은 타당하다. 따라서 남은 선택지는 애플리케이션 계층 강제를 빠짐없이 적용하는 것뿐이며, 그것이 이 절의 결함들이 더 중요해지는 이유다.

구조적 제약도 있다 — `sale_items` 에는 `storeId` 컬럼 자체가 없어, DB로 경계를 걸려면 컬럼 추가부터 필요하다.

4-2. 커버리지 지표

항목	수치	해석
쓰기 라우트 (POST/PUT/PATCH/DELETE)	461	—
@FunctionGuard 적용	198	약 43%. 2026-07-28 하루에 크게 늘었다
@Permission (권한 v2)	11	v2는 사실상 미이행 — v1/v2 이중 체계가 공존
@Audit 적용	110	쓰기 라우트의 약 24%만 감사 기록됨
세션 가드(SessionGuard) 적용	3	전역이 아님 — attendance·shared-folders 3개 컨트롤러뿐

4-3. 기업급 통제 — 전부 부재

항목	상태	비고
SSO (OIDC/SAML)	없음	중견 고객 조달 요건에서 자주 요구된다
MFA / 2FA	없음	온보딩 OTP는 신규 매장 가입 검증용이지 로그인 MFA가 아니다
SoD (직무분리)	미설립	승인 임계값 모델은 있으나 자가 승인 차단이 없다(<code>approval.service.ts:168</code>) — maker-checker가 성립하지 않는다
계정 잠금 / 실패 횟수	없음	방어는 레이트리밋(계정 15/분)뿐
비밀번호 정책	최소 6자	복잡도·이력·만료 없음
감사로그 보존 정책	없음	<code>slow_query_log</code> ·에러버퍼에는 <code>prune</code> 이 있으나 <code>audit_logs</code> 에는 없다. 불변성(WORM) 강제도 없다
시크릿 매니저	없음	<code>.env</code> 기반. 실 <code>.env</code> 는 git에 없음을 확인(양호)
앱→DB TLS	미설정	<code>dialectOptions.ssl</code> 설정 0건
CORS	전면 개방	<code>main.ts:55</code> <code>origin: true</code> + <code>credentials: true</code> — 모든 Origin 반사
보안 헤더(helmet 등)	없음	HSTS·CSP·X-Frame-Options 없음
PII 보호	평문	CUIT/DNI·전화·이메일 평문 저장. 컬럼 암호화는 MP 토큰 1종만(AES-256-GCM)

권한 가드의 fail-open 2건. `function-permission.guard.ts:88` — DB에 function slug가 없으면 통과.
`permissions/guards/permission.guard.ts:60-63` — 메타데이터가 없으면 통과(레거시 호환 명시). 권한 데이터가 유실되면 시스템이 잠기는 게 아니라 열린다. 최근 "권한 재시드 slug 기반 전환"이 이 위험을 줄였으나, 기본값 자체는 여전히 열림이다.

근거: 코드 직접 확인 — `audit-log.controller.ts`, `users.service.ts`, `main.ts`, `migrations/*.sql`, `docs/db-risk-analysis-20260727.md:105`

05 가용성 · 재해복구 · 관측

가장 큰 구멍은 성능이 아니라 "장애를 아무도 감지하지 못한다"는 것이다. 헬스체크 엔드포인트가 없고, 외부 uptime 감시가 없고, 인프라 지표 알람이 없다. 현재 유일한 알람은 요청이 500을 났을 때 울리는 Telegram이므로, 프로세스가 죽거나 DB가 내려가면 알림이 오히려 멈춘다. 2026-07-25 재부팅 후 2시간 무중단 다운을 놓친 것이 그 증거다(그 사고 뒤에야 `restart: always` 가 추가됐다).

5-1. 구현 vs 계획 — 정직한 구분

영역	실제 구현된 것	계획만 있는 것
가용성	PM2 cluster 4워커, socket.io Redis 어댑터, <code>restart: always</code>	2번째 API 컨테이너, nginx LB, read replica, 워커 프로세스 분리 서버 2호기는 보류로 결정됨
재해복구	일 1회 pg_dump(03:17, 14일 보관), rclone 오프 사이트 스크립트, 2026-07-26 스테이징 복원 1회 성공	WAL 아카이빙/PITR, 시간별 백업, 월 1회 복원 리허설, RPO/RTO 정의, DR runbook
관측	winston 파일 로그(시크릿 마스킹), slow query 100ms 적재, pool 60초 모니터, <code>/api/diagnostics</code> 3종, Telegram 500 알람, <code>store_error_log</code>	로그 JSON화, <code>/metrics</code> (prom-client 미설치), Prometheus/Grafana/Loki, 알람 규칙, pg_stat_statements, 헬스체크, SLO 정의
다중서버 대비	Redis 중단 fallback 3종, keep-alive 튜닝(65s)	Redis 분산락(현재 cron 리더 가드는 <code>NODE_APP_INSTANCE===0</code> 뿐 — 컨테이너를 2대로 늘리면 cron이 중복 실행된다), graceful shutdown, Redis 공유 캐시
배포	Jenkins 잡(서버에만 정의), 상태 폴링 스크립트	파이프라인 as-code, 롤백 절차, 마이그레이션↔코드 순서 자동 보장

5-2. 수치로 본 위험

RPO (복구 시점 목표) 24시간 일 1회 dump뿐, <code>archive_mode=off</code> . 디스크 장애 시 하루치 판매 소실	RTO (복구 시간 목표) 미정의 저장소 전체에 "RTO" 문자열 0건	복원 리허설 0회 정식 DR 리허설 없음(스테이징 복원 1회는 부하시험 목적)	장애 주입 시험 미실행 워커 kill·Redis 중단·PG 재시작 시 생존/복구 시간 미측정
---	--	--	--

5-3. graceful shutdown 부재

`app.enableShutdownHooks()` 호출이 소스 전체에 없다. 따라서 SIGTERM을 받아도 `database.module.ts` 의 pool 정리 혹은 트리거되지 않고, 진행 중인 요청을 빼내는 드레이닝 장치도 없다. 배포·재시작마다 진행 중인 판매 요청이 끊길 수 있다. 무중단 배포의 전제가 빠져 있는 상태다.

5-4. 확인된 처리 한계 (실측)

축	한계	병목
300매장 분산 판매	100건/s에서 P95 57ms, 실패 0%	병목 미도달 — 목표 대비 6배 여유

단일 매장 판매	개선 전 16건/s 붕괴 → 현재 41건/s	상품 재고 행 락 경합 → pgbouncer 슬롯 포화 → 판매 500 연쇄. CPU가 아님
로그인	60~70건/s , 80건/s부터 붕괴(P95 4.9s)	CPU 바운드 — bcrypt + 권한 조립 + 55KB JSON 직렬화 (47KB가 권한 구조). 3,000터미널 전원 로그인 ≈ 50초

부하 테스트가 **운영 서버** 안에서 수행된다는 점은 남아 있는 위험이다(완화책은 심야 실행 + watchdog).

5-5. 문서와 코드의 불일치

항목	불일치 내용
CLAUDE.md pool 설정	문서는 "min=10, max=80"이라 기술하나 실제 코드는 min=2, max=20 (database.module.ts:58-59). 4워커 전환 시 조정된 값이 문서에 반영되지 않았다
.planning/ROADMAP.md	Phase 64가 "0/10 executed"로 표기돼 있으나 실제로는 W1~W9 완료·운영 배포됨
.planning/STATE.md	최종 갱신 2026-07-24, "Current focus: Phase 61"로 정지 — Phase 63·64를 반영하지 않음
.planning/intel/	DB 스키마 intel이 2026-06-25 자. Phase 64 W10의 재생성 작업이 미완
DATABASE_SCHEMA.md	Stocks 정의가 3컬럼(id/stock/productBranchId)으로 남아 있어 type·note·operationDate·isActive 누락

근거: ecosystem.config.js , main.ts , database.module.ts , docs/DROPBOX_BACKUP_SETUP.md , docs/db-risk-analysis-20260727.md , .gsd/spec-phase54/60/63 , loadtest/README.md

06 중견 제조·유통 ERP 대비 계획·분석 기능

거래를 기록하는 기능은 넓고 깊다. 반면 그 기록을 근거로 **미래를 계획**하는 계층은 거의 비어 있다. 아래는 키워드 전수 검색으로 확인한 결과다.

먼저 정정할 것 하나. 이전 보고서가 "관리자가 사후 보고에 의존"한다고 평가했으나, 실측 결과 **운영 예외함은 이미 존재한다** — Centro de Control (GET /dashboard/admin/control-center). 위젯 8종(금전함·외주·자재·미수금·발송지연·수표·인프라·매출맥박)을 단일 호출로 모아 urgente / hoy / info 3단계 심각도와 딥링크로 제공한다. 이전 평가보다 나은 지점이다.

기능	상태	확인 내용
안전재고 · 재주문점	원자재만	원자재는 있다 — mes_materials.minStock (리오더 포인트)에 더해 30일 소비율 기반 고갈 예상 일(D-7 경보)까지 산출한다. 완제품은 없다 — 상품 모델에 최소·안전재고 컬럼이 없고, 재고 부족 알림은 전 매장·전 SKU 공통 고정 임계값 5다 (reportsAlertas.service.ts:11 , 코크핏 버전에도 동일 값 중복). 회전이 빠른 SKU와 느린 SKU가 같은 기준으로 경보된다.
리드타임 · EOQ	없음	저장소 전체 매치 0건. 공급업체 모델의 결제조건은 자유 텍스트이고 leadTimeDays 가 없다
발주(PO) · 자동발주	없음	발주 라우트 0건. 가장 근접한 것은 자재 입고 기록(type='ENTRADA')인데 이는 이미 도착한 것이라 미착(on-order) 수량 개념이 없어 재주문 계산 자체가 성립하지 않는다
수요예측	없음	가장 근접한 것은 시즌 회전을 리포트인데, 코드 주석이 스스로 "평균재고 데이터가 없어 현재재고를 proxy로 사용"이라 밝히고 있다 — 단순 나눗셈 소진일이지 예측 모델이 아니다
이상탐지	부분	지점별 오늘 매출을 같은 요일 4주 평균과 비교해 -30% 이하면 경보. 급락만 감지하고 급증은 감지하지 않으며, 지점 단위라 SKU별이 아니다
재고 실사	단건만	POST /reports/stocks-cockpit/adjust 로 SKU 1건씩 실사값 입력·차이 반영은 된다. 실사 세션·대상 목록 생성·재고 동결·2인 카운트·차이 승인·순환실사 스케줄은 없다
ABC 분석	없음	파레토·누적기여도 산출 코드 0건
고객 RFM	원시값만	일 1회 사전집계 크론이 실재한다 — 최근구매일·구매횟수·구매총액을 client_segments 에 UPSERT 하고 조건 필터를 제공한다. 다만 R/F/M 점수(1-5)도, "Champions/At Risk/Lost" 같은 세그먼트 라벨도 없다. 스코어링이 아니라 원시지표 필터다
CLV · 이탈위험 · 로열티/쿠폰	없음	누적 구매액(과거값)은 있으나 미래가치·이탈확률 없음. 프로모션은 상품 단위라 쿠폰 코드·고객별 발급/사용 이력이 없다
캠페인 · 세그먼트	양호	다채널(WhatsApp·Email) 발송 크론, 세그먼트 빌더, 수신거부 토큰, 웹훅, 발송 전 대상 수 미리보기까지 실무 사용 가능한 완성도 . A/B 테스트·전환 어트리뷰션은 없음
MRP(자재소요)	단건 전개만	Cut Ticket이 로트 1건의 BOM을 감모율 포함해 전개하고 재고 신호등(OUT/LOW/SUFFICIENT)을 준다. 없는 것은 MRP의 핵심 — 순소요 네팅(가용·미착 차감), 시간축 전개, 계획오더 자동 생성
Capacity · 예상 완료일	부하 가시성만	공정(etapa)별 대기 수량 백로그는 보여주나 능력(시간/CAPA) 모델 자체가 없어 부하율을 낼 수 없다. ETA 계산 0건 — Cut Ticket의 dueDate 는 코드에 null 하드코딩 + "확장 예정" 주석
납기 관리(외주)	구현됨	발송 납기일 + 매일 08:00 지연 경보 크론 + ON_TRACK / AT_RISK / LATE 상태

로트 추적성	양 끝이 끊김	외주 내부는 FK로 연결된다 — 로트 → 발송 → 수령 → QC → 재작업. 그러나 ① 상류: 자재에 lot/batch 컬럼이 없어 "어느 원단 롤을 썼는가"를 답할 수 없고, ② 하류: 완제품 입고가 자유 텍스트 note로만 로트를 남기며 <code>sale_items</code> 에 <code>loteId</code> 가 없다(직접 확인). → 리콜 시 "이 로트가 누구에게 팔렸나"를 질의할 수 없다.
품질 KPI	불량률만	불량률은 있다 — 12주 주간 추이 + 업체별 집계 + 불량코드 마스터. 재작업률은 집계 가 없고(재작업 오더 자체는 존재), 사내 생산(MES)의 <code>defectQuantity</code> 는 저장만 되고 집계 사용처가 0건이다
BI · 리포트	강함	코크핏 17종 + 대시보드 6종, <code>/reports/*</code> 라우트 100개 이상, Excel 내보내기 15개 라우트. 단 ① 필터가 리포트별 하드코딩 세트라 임의 컬럼·피벗 정의 불가, ② PDF는 "próximamente" 알림, ③ 예약 리포트는 크론이 주석 처리된 빈 스텝
운영 예외함	구현됨	위 정정 참조. 남은 한계 3가지: ① 읽기 전용 피드 — 확인처리·담당자 배정·스누즈가 없어 "할일함"이 아니다(관련 코드 0건), ② 완제품 품질이 빠져 있다 — 원자재 고갈만 포함, ③ <code>failed outbox</code> 를 볼 수 없다 — <code>pending</code> 건수만 집계하고 실패 잡 목록·재처리 엔드포인트가 없다
반품 · 교환	제한적	반품이 판매 전체 무효화(<code>nullifySale</code>) 단일 경로다. 부분 반품·교환 전용 흐름이 없다(관련 코드 0건). 의류 소매에서 사이즈 교환은 일상 업무이므로, 전체 취소 후 재판매로 우회하면 매출·재고 통계가 왜곡된다.

패턴이 하나 보인다. 외주(talleres) 도메인은 로트·QC·재작업·불량률·납기경보까지 **중견 제조 수준으로 깊다**. 반면 같은 개념이 사내 생산(MES)과 소매 재고에는 알계만 적용돼 있다 — 자재에 lot이 없고, 완제품에 안전재고가 없고, MES 불량 데이터는 집계되지 않는다. 새로 만들 것보다, 외주에서 이미 검증된 패턴을 나머지 두 도메인으로 옮기는 편이 비용이 낮다.

포지셔닝 판단은 이전 보고서와 동일하다. 범용 ERP와 전면 경쟁하기보다 다중매장 패션·도소매 + 외주생산 + 현장 하드웨어 + 지역 결제·세무가 한 흐름으로 연결되는 제품이라는 차별점은 여전히 유효하고 강하다. 다만 계획 기능은 "차별화"가 아니라 "중견 고객이 당연히 있다고 가정하는 것"이므로, 완제품 안전재고·발주·부분반품 정도는 차별화 기능이 아니라 기본기로 분류해야 한다.

근거: 소스 전수 실측 — `dashboard-admin.service.ts`, `materials.model.ts:83`, `reportsAlertas.service.ts:11`, `segment-refresh.cron.ts`, `cut-ticket.types.ts:73`, `sales-item.model.ts`, `reports.schedule.ts`, `dashboard-v2.service.ts:141-183`

07 잔여 과제 — 우선순위와 근거

등급	과제	방치 시 결과	판단 근거
P0	재고 원장 단일화 원장 불변 전면 적용 + 이동/폐기 의 캐시 갱신 + 이동유형 표준화	이동·폐기 1건마다 캐시-원장 괴 리 누적. 재고 화면 숫자를 아무도 믿지 못하게 된다	원장 변경 코드 9곳 잔존, 캐시 미갱신 경로 3종, 유형값 2종이 리포 트를 오염
P0	재고 정합성 자동 대조·보정 야간 크론 + 1회 백필	드리프트를 발견할 수단이 수동 SQL뿐. 문제를 사후에도 못 찾는 다	재고 관련 크론 0건. 동형 장치가 MP 지갑에 이미 있어 이식 비용 낮음
P0	테넌트·감사 경계 통합 감사로그 2건 + 사용자 관리 3건	타 매장 감사로그·사용자 조작 가 능. 중견 고객 보안 심사에서 즉시 탈락	코드 직접 확인. fail-open이 "차단"이 아니라 "전체 공개"로 귀결
P0	자격증명 회전 마이그레이션 10개 파일 정리	저장소 접근자 전원이 운영 DB 비 밀번호를 보유	git 추적 상태 확인 완료
P1	장애 감지 체계 헬스체크 + 외부 uptime + 인프 라 알람	서비스 다운을 고객 전화로 알게 된다. 실제로 2시간 다운을 놓친 이력	헬스체크 라우트 0건, 외부 감시 없음, Telegram은 500에만 반응
P1	DR 계약 수립 RPO/RTO 정의 + 월 1회 복원 리 허설 + WAL 아카이빙	디스크 장애 시 하루치 판매 소실, 복구 시간 예측 불가	RPO 24h 실측, RTO 문자열 0건, 정식 리허설 0회
P1	graceful shutdown	배포·재시작마다 진행 중 판매 요 청 유실 가능	<code>enableShutdownHooks</code> 호출 0건
P1	Phase 64 마감 W10 + 브라우저 UAT + 계획 문 서 동기화	완료 여부를 아무도 확인할 수 없 다. 다음 작업의 기준점이 흔들린 다	UAT 12건 전부 공란, ROADMAP·STATE·intel 불일치
P2	부분 반품·교환	사이즈 교환을 전체 취소+재판매 로 우회 → 매출·재고 통계 왜곡	전용 흐름 0건
P2	안전재고·재주문점·발주	품절·과잉재고를 감으로 관리	관련 코드 0건. 단 P0(재고 정합성) 완료 후에 착수해야 한다
P2	운영 예외함	문제를 찾으려 여러 화면을 돌아 다녀야 한다	알림 인프라(Telegram·소켓)는 이미 있어 조립 비용이 낮다
P3	SSO·MFA·SoD, 셀프서비스 BI, 개발자 포털	중견 고객 조달 요건 미충족	영업 파이프라인에 실제 요구가 생길 때 착수

순서에 대한 원칙 하나. 이전 보고서의 "새 화면 수보다 데이터 오류 가능성을 먼저 줄인다"는 원칙은 지난 이틀간 정확히 지켜졌고 효과를 냈다. 같은 원칙을 한 번 더 적용하면, 다음 Phase는 재고 정합성이어야 한다. 자동발주·수요예측은 정확한 재고 잔액을 입력으로 삼는 기능이므로, 잔액이 틀린 상태에서 만들면 틀린 발주를 자동으로 내는 시스템이 된다.

08 다음 Phase 제안 — Phase 65

Phase 65: 재고 원장 단일 진실 · 테넌트/감사 경계 통합 · 장애 감지

Phase 64가 "판매 한 건이 두 건이 되는 것"을 막았다면, Phase 65는 "재고 숫자가 조용히 들어지는 것"과 "들어진 걸 아무도 모르는 것"을 막는다. 신규 기능 0 — 전부 무회귀 교정.

Wave 구성 (제안)

W	내용	완료 판정
W1	이동유형 표준화 — 'ajuste'→'adjust', 'produccion' 을 union 에 등록, 리포트 필터 교정 + 기존 행 백필	모델 union 밖 type 기록 0건(grep 게이트). 코크핏 입고·판매 집계에 보정·생산 행이 섞이지 않음
W2	원장 불변 전면 적용 — productStock.service.ts · products.service.ts · subcon-material-issue.service.ts 의 destroy/update 9곳을 보정 행 방식으로 교체	stocks 대상 destroy/update 전 저장소 0건. 기존 라우트 호환 유지
W3	캐시 갱신 누락 통합 — 이동/폐기·로트 입고·변형 생성이 같은 트랜잭션에서 products.stock 을 조정. 부모 집계를 원장 합 기준으로 전환	실패 주입 시 원장·캐시 동시 롤백. 이동 1건 후 캐시=원장 합
W4	가용재고 정의 확정 — 현재고/예약/가용을 분리한 단일 계산 함수. 재고 SQL 4벌을 그 함수로 수렴	is_active · type 필터가 경로마다 다르지 않음
W5	정합성 대조·보정 크론 — mp-wallet-reconcile 패턴 이식. 예약분을 제외한 정확한 대조 쿼리 + 1회 백필 + 결과를 진단 API로 노출	드리프트 건수가 지표로 관측됨. 임계 초과 시 알람
W6	테넌트·감사 경계 — 감사로그 2개 라우트에 store 스코프 강제(fail-open 제거), 사용자 수정/삭제에 storeId 대조, dto.storeId 이동을 superadmin 전용으로 제한, 자가 승인 차단	타 매장 리소스 접근 시 403. 권한 없음이 "전체 공개"로 귀결되지 않음
W7	자격증명 위생 — 마이그레이션 10개 파일에서 비밀번호 제거, 계정 회전, 시크릿 폴백 리터럴 제거(미설정 시 부팅 실패로 전환)	저장소 내 자격증명 0건. 폴백으로 조용히 동작하는 경로 없음
W8	장애 감지 최소셋 — /health (DB·Redis 체크) + 컨테이너 healthcheck + 외부 uptime 감시 + enableShutdownHooks	프로세스·DB 다운이 60초 내 알림. SIGTERM 시 in-flight 요청 정상 종료
W9	Phase 64 마감 — 브라우저 UAT, intel 재생성, ROADMAP·STATE·DATABASE_SCHEMA 동기화, pool 설정 문서 교정	계획 문서와 코드 상태 일치. UAT 결과 기록

Wave 순서: W1 → W2 → W3 → W4 → W5 (재고 계열은 선형 의존) · W6·W7·W8은 병렬 가능 · W9는 마지막. W5는 W1~W4가 끝나야 대조 결과가 의미를 갖는다.

Phase 65에 넣지 않을 것 (명시적 제외)

- RLS·복합 FK — RLS는 도입 금지로 이미 결정됐고, 복합 FK는 sale_items.store_id 컬럼 추가부터 필요해 범위가 과대하다. 애플리케이션 계층 강제로 대체한다.
- 안전재고·자동발주·수요예측 — 재고 잔액이 정확해진 뒤에 착수한다. Phase 66 이후.
- 서버 2호가·read replica — 이미 보류로 결정됨. W8은 "감지"까지만 다루고 "이중화"는 다루지 않는다.
- SSO·MFA·셀프서비스 BI — 영업 요구가 확정될 때.

완료 게이트 (Phase 65 전체)

지표	목표
원장-캐시 불일치 상품 수	백필 후 0, 이후 야간 크론에서 지속 0 (예약분 제외한 정확한 대조 기준)
stocks destroy/update	전 저장소 0건
모델 union 밖 이동유형	0건
크로스테넌트 접근	감사로그·사용자 경로에서 403. 회귀 테스트로 고정
저장소 내 평균 자격증명	0건
장애 감지 시간	프로세스/DB 다운 → 60초 내 알림
무회귀	Phase 64 동시성 스위트 8종 전부 통과 유지, 부하 25건/s 재측정에서 저장 실패 0

이 제안은 .planning/phases/65-.../ 에 GSD 형식(CONTEXT·SPEC·PLAN)으로 별도 작성됨.

09 최종 제언

지난 이틀은 **잘한 작업**이다. 외부 리뷰 12건을 받아 이틀 만에 코드·마이그레이션·유닛·동시성·부하까지 검증하고 운영에 반영한 것은 중견 규모 조직에서도 흔치 않은 속도다. 특히 **"판매는 커밋했는데 API는 500"**이라는, 겉으로는 잘 안 보이지만 현장에서 가장 비싼 오류를 구조적으로 제거한 것이 핵심 성과다.

세 문장 요약

- 01** 거래는 안전해졌고, 재고는 아직 아니다.
판매·취소·생산·동기화의 원자성과 멍등성은 담렸다. 그러나 재고 잔액은 여전히 두 곳에 따로 살고 있고, 그 둘을 맞춰주는 장치가 없다.
- 02** 보이지 않는 것이 가장 위험하다.
재고 드리프트도, 서비스 다운도, 크로스테넌트 접근도 현재는 **사람이 우연히 발견해야** 알 수 있다. 다음 투자는 새 기능이 아니라 **감지 장치**에 해야 한다.
- 03** 계획 기능은 정합성 위에만 지을 수 있다.
자동발주·수요예측은 정확한 잔액을 전제로 한다. 순서를 뒤집으면 틀린 숫자로 틀린 발주를 자동 생성하는 시스템이 된다.

이 문서에서 확인하지 못한 것 (한계 명시)

- 운영 DB에 직접 접속하지 않았다. 재고 드리프트의 실제 건수·금액 규모는 미측정이다. Phase 65 W5의 첫 작업이 이 측정이어야 한다.
- 브라우저 UAT를 실행하지 않았다. Phase 64 변경의 화면 단 영향은 미확인이다.
- 운영 로그를 열람하지 않았다. print-agent polling과 PM2 4위커 조합에서 `Session ID unknown` 이 실제로 발생하는지는 **추정 단계**이며, 운영 로그 확인이 필요하다.
- 보안 항목은 **정적 코드 분석** 기준이다. 실제 침투 시나리오 검증(권한 우회 실행)은 수행하지 않았다.

분석 근거

- `ACE_online_1.0` 소스 실측 — 판매·재고·생산·outbox·권한·감사 경로, 파일·라인 확인
- 마이그레이션 SQL 218개, `2026-07-28-phase64-missing-constraints.sql` (644줄) 포함
- Phase 63 스케일·복원력 계획 및 k6 부하 측정 기록, Phase 64 SPEC·CONTEXT·PLAN 10건·VALIDATION·LOADTEST
- `docs/db-risk-analysis-20260727.md` (운영 DB 위험 분석, 2026-07-27)
- 빌드·정적분석 재실행 — `tsc --noEmit` (백엔드 잔여 오류 1건), `eslint` (프론트 29 warning, 0 error, 전부 react-hooks/exhaustive-deps)
- git 이력 2026-07-25 ~ 2026-07-28 (커밋 `b02ac0d` 까지)

본 문서는 제품 전략과 기술 투자 우선순위를 정하기 위한 **내부 진단 자료**다. 실제 운영 변경은 스테이징 동시성 테스트, 데이터 정합성 검사, 운영 승인 절차를 거쳐야 한다. 특히 **자격증명 회전(W7)**과 **재고 백필(W5)**은 되돌리기 어려운 작업이므로 별도 승인과 롤백 계획이 필요하다.